Edge cases (proving gather actually waits) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `gather_practice.py` (Edge cases section): it's easy to assume `gather` hands back results as soon as the first task finishes — this exercise proves, with a real timestamp, that it doesn't. **If not:** the first time a slow straggler silently doubled an agent turn's latency, you'd have no smaller example to recognize the cause from.

Basic Version

Approach 1 — the direct way

```
# gather_practice.py — Edge cases section
import asyncio
import time

async def fast_task():
    await asyncio.sleep(0.1)
    print("fast_task finished at", time.perf_counter())
    return "fast"

async def slow_task():
    await asyncio.sleep(5)
    print("slow_task finished at", time.perf_counter())
    return "slow"

async def main():
    start = time.perf_counter()
    print("starting at", start)
    results = await asyncio.gather(fast_task(), slow_task())
    print("gather returned at", time.perf_counter())
    print("results:", results)

asyncio.run(main())
```

Expected output (your exact numbers will vary slightly):

```
starting at 1000.00
fast_task finished at 1000.10
slow_task finished at 1005.00
gather returned at 1005.00
results: ['fast', 'slow']
```

`fast_task`'s own print line shows up almost immediately, but nothing after the `gather(...)` line runs until the slow task also finishes, a full 5 seconds later. That gap is the whole proof: `gather` doesn't hand back control as soon as the first task is done, it waits for every task it was given.

This version works correctly. It's missing type hints and an `if __name__ == "__main__":` guard — both fine for a first working version.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — typed, with a `__main__` guard

```
# gather_practice.py — Edge cases section
import asyncio
import time


async def fast_task() -> str:
    await asyncio.sleep(0.1)
    print(f"fast_task finished at {time.perf_counter():.2f}")
    return "fast"


async def slow_task() -> str:
    await asyncio.sleep(5)
    print(f"slow_task finished at {time.perf_counter():.2f}")
    return "slow"


async def main() -> None:
    start = time.perf_counter()
    print(f"starting at {start:.2f}")

    results = await asyncio.gather(fast_task(), slow_task())

    print(f"gather returned at {time.perf_counter():.2f}")
    print(f"results: {results}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output:

```
starting at 1000.00
fast_task finished at 1000.10
slow_task finished at 1005.00
```

```
gather returned at 1005.00
results: ['fast', 'slow']
```

Difference from Basic: full type hints (`-> str` on both tasks, `-> None` on `main`) document each coroutine's contract without reading its body. The `if __name__ == "__main__":` guard means this file's functions can be imported and reused (say, from a test checking `gather` 's timing) without the demonstration run firing automatically on import.

Approach 2 — a genuinely unbounded task, capped with its own timeout

Story: `slow_task` above is slow but always, eventually, bounded at 5 seconds — a real task might never come back at all. **If not:** a single hung network call inside a `gather` group would hold every other, already-finished result hostage indefinitely.

```
# gather_practice.py — Edge cases section
import asyncio
import time


async def fast_task() -> str:
    await asyncio.sleep(0.1)
    return "fast"


async def truly_slow_task() -> str:
    await asyncio.sleep(60)   # stands in for "might never come back"
    return "finally done"


async def main_bounded() -> None:
    start = time.perf_counter()

    bounded_slow = asyncio.wait_for(truly_slow_task(), timeout=2)
    results = await asyncio.gather(
        fast_task(), bounded_slow, return_exceptions=True,
    )

    print(f"whole group finished in {time.perf_counter() - start:.2f}s")
    for result in results:
        if isinstance(result, Exception):
            print("a task timed out or failed:", repr(result))
        else:
            print("a task succeeded:", result)
```

```
if __name__ == "__main__":
    asyncio.run(main_bounded())
```

Expected output (roughly 2 seconds, not 60):

```
whole group finished in 2.00s
a task succeeded: fast
a task timed out or failed: TimeoutError()
```

`asyncio.wait_for(truly_slow_task(), timeout=2)` is created *before* the call to `gather` — it's what actually cancels `truly_slow_task` after 2 seconds and turns that into a `TimeoutError` sitting in `bounded_slow`'s slot, instead of `gather` waiting the full 60 seconds like it did for `slow_task` in Approach 1.

Approach 3 — the same fix applied to a whole list of tasks, not just one

Story: Approach 2 caps one risky task — a real agent step usually fans out to several independent sources at once, and any one of them could be the slow or dead one. **If not:** the fix from Approach 2 would need re-deriving by hand every time the group's size changed, instead of generalizing to one loop that wraps every task.

```
# gather_practice.py — Edge cases section
import asyncio


async def fetch(source_id: int, delay: float) -> str:
    await asyncio.sleep(delay)
    return f"result from source {source_id}"


async def fetch_all(
    sources: list[tuple[int, float]], per_task_timeout: float,
) -> list:
    bounded_tasks = []
    for source_id, delay in sources:
        bounded_tasks.append(
            asyncio.wait_for(fetch(source_id, delay), timeout=per_task_timeout)
        )
    return await asyncio.gather(*bounded_tasks, return_exceptions=True)


async def main() -> None:
    # 3 fast sources, 1 that would hang for a very long time
    sources = [(1, 0.2), (2, 0.3), (3, 0.1), (4, 120)]
    results = await fetch_all(sources, per_task_timeout=3)
```

```
    for i in range(len(sources)):
        source_id = sources[i][0]
        result = results[i]
        if isinstance(result, Exception):
            status = "timed out"
        else:
            status = result
        print(f"source {source_id}: {status}")


if __name__ == "__main__":
    asyncio.run(main())
```

Expected output (finishes in about 3 seconds total, not 120):

```
source 1: result from source 1
source 2: result from source 2
source 3: result from source 3
source 4: timed out
```

This is the same pattern as Approach 2, generalized: wrap *every* task in `asyncio.wait_for(..., timeout=...)` before handing the whole list to `gather(*tasks, return_exceptions=True)`. One slow or dead source costs the group exactly `per_task_timeout` seconds, not however long that one source decides to take.

Difference from Approach 1, and between Approaches 2/3: Approach 1's `slow_task` is slow but always, eventually, bounded at 5 seconds — the exercise proves `gather` waits for it, but never asks what if it didn't come back at all. Approach 2 answers that for a single task. Approach 3 shows the same fix scales cleanly to any number of tasks with one simple loop, which is the shape a real "search 4 independent sources and combine what comes back" agent step actually takes.

Which one should you actually write? Any time you're `gather`-ing calls to something outside your program's control — an API, a database, a vector store — wrap each individual call in `asyncio.wait_for(..., timeout=...)` before it joins the group, the way Approach 2 and 3 both do. Skip it only for tasks you've deliberately bounded some other way already (like `asyncio.sleep(2)` in the earlier exercises, which can never hang). Combined with `return_exceptions=True` from the `gather_speed` exercise, this is what turns "one slow or dead source can freeze the whole group forever" into "the group always finishes within a time you chose, with each source's outcome clearly labeled."